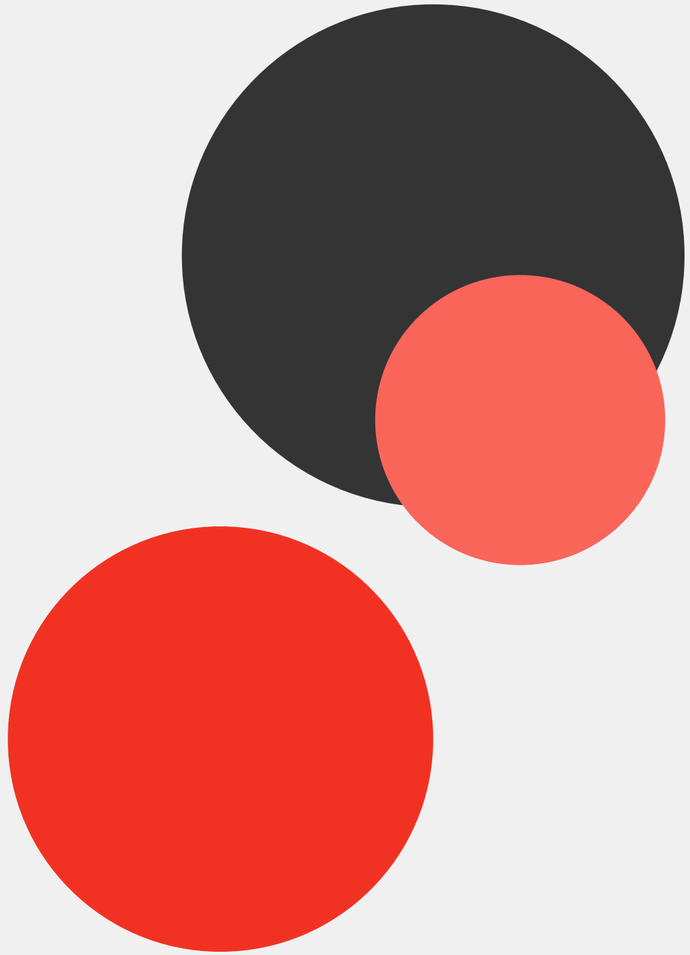
SEMINARIO DE ACTUALIZACIÓN · IFTS N.º 18

Análisis de sentimientos de opiniones sobre películas

Comparación de modelos preentrenados y despliegue de una aplicación web



Lucas Suárez · Eric Silvestri · Walter Willich

Ciclo lectivo 2026 · Entrega: 16 de octubre

tp-integrador-sentimientos.streamlit.app

¿Qué problema resolvemos?

ENTRADA

Una opinión breve escrita por el usuario.

PROCESAMIENTO

Un modelo preentrenado analiza el texto.

SALIDA

Etiqueta de sentimiento y puntaje del modelo.

¿Por qué es difícil?

Lo que nos dificultó fue que el lenguaje no siempre es literal, muchas frases están compuestas de ironía, negaciones, jerga, frases que mezclan opiniones, y esto puede confundir al modelo.

Dos modelos

DistilBERT (version mas rapida de Bert ajustada especificamente para clasificacion de sentimientos utilizando el data set SST-2)

Procesamiento Bidireccional Simultáneo: No leen de izquierda a derecha de forma aislada.

Mecanismo de Atención: Analizan todas las palabras de la frase juntas para entender el significado exacto según el contexto que las rodea.

-Clasificación binaria: positivo / negativo.

-Idioma Ingles

Bert, es una arquitectura de red neuronal desarrollada por Google, enfocada en el procesamiento de textos

XLM-RoBERTa (es una versión avanzada y multilingüe de la arquitectura roberta, entrenado y ajustado **con un dataset de publicaciones de Twitter multilingües**)

Procesamiento Bidireccional Simultáneo: No leen de izquierda a derecha de forma aislada.
Mecanismo de Atención: Analizan todas las palabras de la frase juntas para entender el significado exacto según el contexto que las rodea.

-Puede devolver: positivo / neutral / negativo.

-Multilingüe.

Roberta, es una arquitectura de red neuronal desarrollada por meta Facebook, enfocada en el procesamiento de textos

Resultados de la comparación

DistilBERT

6 de 8 aciertos · 75 %

XLM-RoBERTa

6 de 8 aciertos · 75 %

Casos que muestran las limitaciones

Frase	Esperado	DistilBERT	XLM-R
“Yeah, great movie... I almost fell asleep.”	Negativo	Negativo ✓	Positivo ✗
“This movie was sick!”	Positivo	Negativo ✗	Negativo ✗
“Meh.”	Neutral	Negativo ✗	Neutral ✓

Decidimos avanzar con DistilBERT, ya que clasifica solamente positivo o negativo y esta enfocado en un solo lenguaje

Pipeline: inferencia local y tiempos

Primera ejecución

Utilizamos pipeline de la librería Transformers.

El pipeline recibe directamente el texto ingresado por el usuario y en conjunto con el data set corpus sst2 4 y al modelo, el sistema web devuelve una estructura de datos lista para usar

DistilBERT · 8 frases

0,3697 s total
0,0462 s promedio por frase.

XLM-R · 8 frases

0,6581 s total
0,0823 s promedio por frase.

¿Qué devuelve pipeline?

El modelo Retorna una lista que contiene un diccionario. Para extraer el resultado en Python, se accede a las claves con: resultado[0]["label"] para obtener la etiqueta (ej. POSITIVE o NEGATIVE). Resultado[0]["score"] para obtener el puntaje o probabilidad de confianza.

Las mediciones dependen del entorno y no son una garantía de rendimiento en producción.

InferenceClient y errores reales

InferenceClient · HTTP 402

El proveedor indicó que no quedaban créditos de inferencia, por eso no pudimos completar la solicitud ni medir una latencia remota válida.

Otros errores

Problemas de carga/tokenizador en un checkpoint inicial; HTTP 401 al validar credenciales; rechazo GH013 por detección de tokens en el historial de GitHub.

Decisión

Elegimos pipeline para la aplicación porque ejecuta el modelo en el entorno de Streamlit y no depende de créditos de un proveedor remoto. A cambio, requiere memoria y recursos suficientes para cargar los pesos.

Los tokens son credenciales: no deben guardarse en el código ni publicarse en el repositorio.

Arquitectura y despliegue

1 · Usuario

Escribe una opinión desde el navegador.

2 · Streamlit Cloud

Ejecuta app.py y calcula la predicción.

3 · Hugging Face Hub

Aloja los pesos que se descargan al iniciar.

¿Por qué Streamlit Community Cloud?

Permite publicar una interfaz Python conectada al repositorio de GitHub.
Render y Hugging Face Spaces son alternativas; se deben comparar memoria, arranque en frío, cuotas y límites.
El primer acceso puede tardar por el arranque o la descarga del modelo.

Conclusiones y próximos pasos

Qué logramos

Comparamos dos modelos, inspeccionamos salidas, medimos inferencia y desplegamos una app.

Qué aprendimos

Una etiqueta y un score no garantizan comprensión de ironía, jerga o frases mixtas.

Qué falta

Ampliar la muestra, probar más condiciones y medir memoria, latencia y concurrencia.

Aplicación publicada

tp-integrador-sentimientos.streamlit.app

Repositorio GitHub

github.com/Lukit4as/portfolio-seminario

Conclusión: el contexto importa; el score no garantiza una interpretación correcta.



FIN

